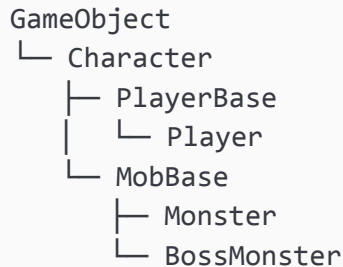


08 장 학습 과제

과제 1. 게임 객체 계층 구성

아래 조건에 맞는 클래스 계층을 작성한다.



- **GameSystem**은 프레임 시간을 제공하고 **Update()**는 인수 없는 공통 인터페이스로 유지
- **GameObject**는 **Update()**와 **Render()**를 순수 가상 함수로 선언
- **Character**는 체력과 피해 처리 기능 제공
- **PlayerBase**는 **ProcessInput()** 계약 제공
- **MobBase**는 **Think()**와 **Attack()** 계약 제공
- 구체 클래스는 모든 순수 가상 함수를 구현
- 모든 재정의 함수에 **override** 사용
- 기반 클래스 소멸자는 가상 소멸자로 선언

Mermaid 클래스 다이어그램도 함께 작성한다.

과제 2. 생성과 소멸 순서 추적

GameObject, **Character**, **MobBase**, **BossMonster**의 생성자와 소멸자에서 클래스 이름을 출력한다.

```
{
    BossMonster boss;
}
```

출력 결과를 기록하고 생성과 소멸 방향을 설명한다. 각 클래스에 데이터 멤버 객체를 하나씩 추가하여 기반 클래스와 멤버의 초기화 순서도 확인한다.

과제 3. 이름 숨김과 **using** 선언

Character에 두 개의 **Move()** 오버로드를 작성한다.

```
void Move(float x, float y);
void Move(float distance);
```

Player에 **Move(int direction)**을 추가한 뒤 기반 함수가 숨겨지는 현상을 확인한다. **using Character::Move;**를 적용하여 세 함수를 모두 호출할 수 있도록 수정한다.

과제 4. 가상 함수와 기본 인수

GameObject::Draw(int layer = 0)과 Player::Draw(int layer = 10)을 작성한다.

```
Player player;  
GameObject& object = player;  
player.Draw();  
object.Draw();
```

두 호출에서 실행되는 함수와 전달되는 기본 인수가 어떻게 결정되는지 설명한다. 가상 함수마다 다른 기본 인수를 두지 않는 개선 코드도 작성한다.

과제 5. 다형 객체 복제

GameObject에 가상 Clone()을 추가하고 Player, Monster, BossMonster에서 구현한다.

```
virtual std::unique_ptr<GameObject> Clone() const = 0;
```

원본 객체를 복제한 뒤 원본의 상태만 변경하여 복사본이 독립적인지 확인한다. 복제 결과의 동적 타입이 유지되는지도 가상 함수 호출로 확인한다.

과제 6. RTTI 사용 코드 개선

구체 타입을 검사하는 코드를 작성한다.

```
void ExecuteAction(GameObject& object);  
dynamic_cast로 Player, Monster, BossMonster를 구분하여 행동을 호출한 뒤, 공통 가상  
함수나 작은 인터페이스를 추가하여 타입 분기를 제거한다. 두 구현의 확장성을 비교한다.
```

과제 7. 다이아몬드 상속

GameObject, Character, NetworkObject, NetworkCharacter로 다이아몬드 상속을 구성한다.

- 일반 상속에서 GameObject 부분이 두 개 생기는 현상 확인
- GetId() 호출의 모호성 확인
- 가상 상속으로 기반 부분을 하나로 통합
- 가장 구체적인 클래스가 가상 기반 클래스를 초기화하는 코드 작성

같은 기능을 NetworkComponent 구성으로 바꾼 설계도 제시한다.

과제 8. 상속에서 구성으로 변경

Monster의 행동을 파생 클래스로 구분한다.

```
Monster  
├─ PatrolMonster  
├─ ChaseMonster  
└─ RangedMonster
```

이 구조를 IMobBehavior와 행동 객체 구성으로 변경한다.

- PatrolBehavior
- ChaseBehavior
- RangedAttackBehavior

실행 중 행동 객체를 교체할 수 있도록 구성하고 상속 방식과 클래스 수, 결합도, 테스트 방법을 비교한다.

과제 9. 게임 장면 다형 객체 관리

GameScene이 `std::vector<std::unique_ptr<GameObject>>`를 소유하도록 작성한다.

필수 기능:

- Player, Monster, BossMonster 추가
- 전체 객체 갱신과 렌더링
- 식별자로 객체 검색
- 인덱스로 객체 복제
- 장면 소멸 시 모든 객체 자동 해제
- 기반 클래스 포인터 삭제가 안전하도록 가상 소멸자 적용

원시 포인터를 소유하지 않으며 객체 잘림이 발생하지 않아야 한다.

